

Оглавление

1	Теоретические основы	2
1.1	Постановка задачи	2
1.2	Частные случаи раскраски	3
1.2.1	Полный граф	3
1.2.2	Двудольный граф	3
1.2.3	Дерево	4
1.2.4	Цикл	4
1.2.5	Планарный граф	5
2	Жадные алгоритмы для раскраски	7
2.1	Алгоритм последовательной раскраски (Greedy)	7
2.2	DSatur (Degree of Saturation)	9
2.3	Эксперименты	13
3	Конструктивные алгоритмы	18
3.1	RLF	18
3.2	Эксперименты	21
4	Точные алгоритмы	27
4.1	Перебор вершин	27
4.1.1	Отсечения по числу цветов	29
4.1.2	Выбор следующей вершины	29
4.1.3	Рационализация	29
4.1.4	Реализация	31
4.2	Метод Зыкова	32
	Приложение	35
	Список литературы	48

1 Теоретические основы

1.1 Постановка задачи

Пусть дан неориентированный простой граф $G = (V, E)$.

Определение 1.1. *Вершинная раскраска графа G — это такое отображение $c : V \rightarrow \{1, 2, \dots, k\}$, где $\{1, 2, \dots, k\}$ — множество «цветов», что для любых двух смежных вершин u и v выполняется условие: $c(u) \neq c(v)$.*

Раскраску также можно рассматривать как разбиение вершин на независимые множества:

$$V = V_1 \cup V_2 \cup \dots \cup V_n, \quad V_i \cap V_j = \emptyset \text{ при } i \neq j$$

Определение 1.2. *Хроматическое число графа G , обозначаемое $\chi(G)$ — это наименьшее число k , для которого существует правильная вершинная k -раскраска.*

Теорема 1.1 (Нижняя оценка хроматического числа). *Для любого графа G выполняется неравенство:*

$$\chi(G) \geq \omega(G),$$

где $\omega(G)$ — кликовое число графа (размер наибольшей клики).

Доказательство. Все вершины клики размера $\omega(G)$ попарно смежны, следовательно, для их раскраски требуется не менее $\omega(G)$ цветов. $\square$

1.2 Частные случаи раскраски

1.2.1 Полный граф

Полный граф из n вершин, обозначается K_n . Содержит $n * (n - 1) / 2$ ребер. Очевидно, что для его раскраски необходимо ровно n цветов, так как в противном случае нашлось бы 2 вершины, не соединенные ребром. $\chi(K_n) = n$

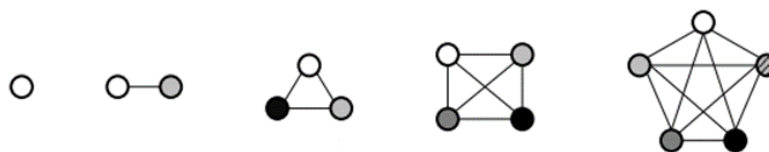


Рис. 1.1

1.2.2 Двудольный граф

Утверждение 1.1. *Непустой граф является двудольным тогда и только тогда, когда он 2-раскрашиваем (его хроматическое число $\chi(G) = 2$).*

Доказательство. (Необходимость) Множество вершин двудольного графа можно разделить на 2 множества: U – никакая вершина из U не соединена с другой вершиной из U V – никакая вершина из V не соединена с другой вершиной из V $G = (U, V, E)$ Тогда можно все вершины из U можно покрасить в цвет 0, а вершины из V покрасить в цвет 1.

(Достаточность) Из всех вершин 1-го цвета составим 1-ую долю, из вершин 2-го цвета составим 2-ую. □

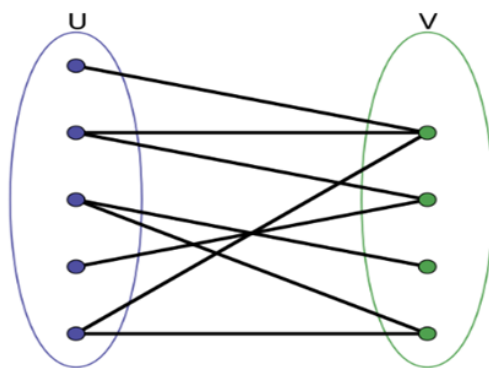


Рис. 1.2: Раскраска двудольного графа

1.2.3 Дерево

Утверждение 1.2. Для дерева $\chi(G) = 2$.

Доказательство. Подвесим дерево за вершину 0. Все вершины на нечетном расстоянии от неё покрасим в цвет 1, все вершины на четном расстоянии (включая и её саму) покрасим в цвет 0.

Докажем, что такая раскраска является корректной. От противного: Допустим, найдутся 2 вершины, соединенные ребром, такие что их цвет совпадает. Тогда, в силу свойств дерева, расстояние от вершины 0 до этих вершин равно d и $d + 1$. Тогда их четность отличается на 1, чего быть не может. $\square$

Следствие 1.1. Дерево является двудольным графом.

Следует из утв. 1 и утв. 2.

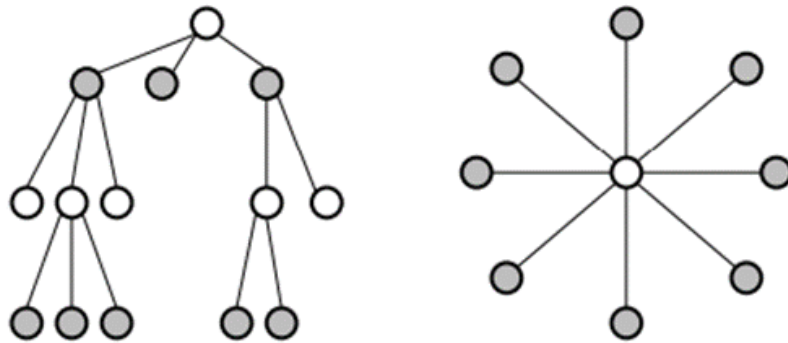


Рис. 1.3: Раскраска дерева

1.2.4 Цикл

Утверждение 1.3. Любой цикл с четным числом вершин можно раскрасить в 2 цвета.

Доказательство. Рассмотрим цикл с четным числом ребер C_{2n} . Без ограничения общности, пусть это будет граф с ребрами вида $E = \{(1, 2), (2, 3), \dots, (2n, 1)\}$. Т.е. вершины с четными номерами соединены с вершинами с нечетными номерами. Тогда его можно будет разбить на 2 независимых множества, т.е. он двудольный. $\square$

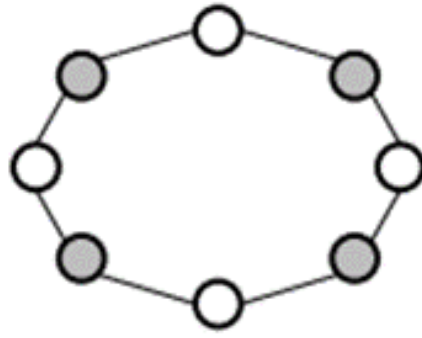


Рис. 1.4: Раскраска цикла с четным числом вершин

Утверждение 1.4. *Любой цикл с нечетным числом вершин можно раскрасить в 3 цвета.*

Доказательство. Докажем, что C_{2n+1} нельзя покрасить в 2 цвета. Рассмотрим цикл $v_1 - v_2 - v_3 - \dots - v_{2n+1} - v_1$. Без ограничения общности, пусть $(v_1) = 0$. Тогда, $(v_2) = 1$ $(v_3) = 0 \dots (v_{2n}) = 1$ $(v_{2n+1}) = 0$ $(v_1) = 1$. Противоречие.

Докажем, что может быть 3 цвета. Поменяем $(v_{2n+1}) = 2$, тогда раскраска является корректной. $\square$

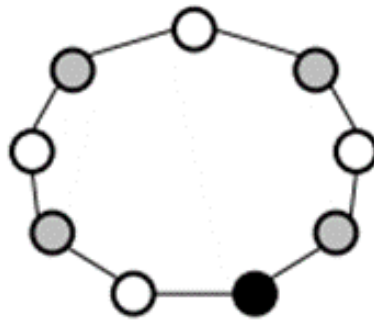


Рис. 1.5: Раскраска цикла с нечетным числом вершин

1.2.5 Планарный граф

Теорема 1.2 (о четырех цветах). *Планарный граф всегда можно покрасить в 4 цвета.*

Доказательство. Доказательство можно посмотреть в [1]. $\square$

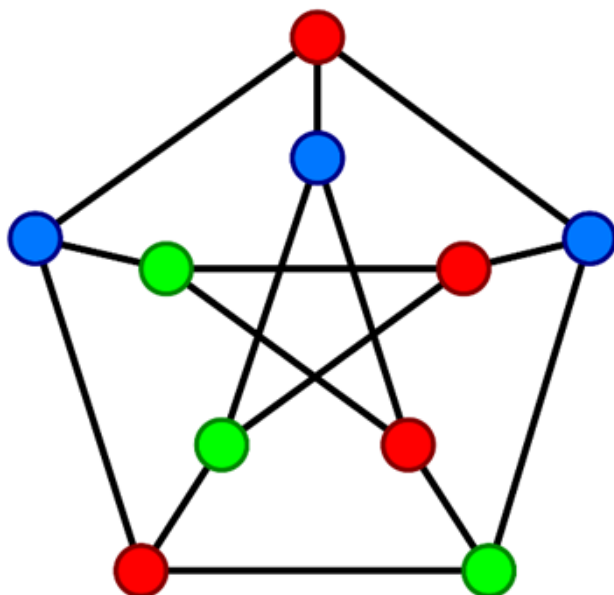


Рис. 1.6: Раскраска планарного графа

2 Жадные алгоритмы для раскраски

Жадными (градиентными) называют алгоритмы, действующие по принципу: максимальный выигрыш на каждом шаге. Такая стратегия не всегда ведет к конечному успеху – иногда выгоднее сделать не наилучший, казалось бы, выбор на очередном шаге, с тем чтобы в итоге получить оптимальное решение. Но в некоторых случаях применение жадных алгоритмов оказывается оправданным.

В частности жадные алгоритмы могут быть использованы в случаях когда можно пожертвовать точностью решения, чтобы получить быстрое решение. Также благодаря ним можно получить верхнюю оценку решения.

2.1 Алгоритм последовательной раскраски (Greedy)

Данный алгоритм – классический представитель класса жадных алгоритмов.

Алгоритм последовательной раскраски — это жадный алгоритм, который раскрашивает вершины графа в порядке их номера, назначая каждой вершине наименьший доступный цвет, не использованный у смежных вершин.

Листинг 2.1: Алгоритм последовательной раскраски

```
1      vector<int> GreedySequential(vector<vector<int>>& graph)
2      {
3          int n = graph.size();
4          vector<int> colors(n, -1);
5
6          for (int vertex = 0; vertex < n; vertex++) {
7              vector<bool> available(n);
8              for (int j : graph[vertex]) {
9                  if (colors[j] != -1)
```

```

9         available[colors[j]] = 1;
10     }
11
12     int min_color = 0;
13     while (available[min_color] == 1)
14         min_color++;
15
16     colors[vertex] = min_color;
17 }
18 return colors;
19 }

```

Рассмотрим вычислительную сложность. Внешний цикл всегда совершает ровно n итераций. Внутри него есть 2 вложенных цикла: число итераций первого равно числу смежных вершин, число итераций второго – не больше. То есть мы гарантированно совершим n итераций и просмотрим $2m$ смежных ребер, то есть совершим $O(n + m)$ итераций. Также можно получить более грубую оценку, если оценить $m \leq n^2$, то есть $O(n^2)$. Такая оценка тоже имеет место быть.

Назовем число цветов, полученное жадным алгоритмом $\chi_g(G)$.

Утверждение 2.1. $\chi_g(G) \geq \chi(G)$

Доказательство. От противного. Допустим, что $\chi_g(G) < \chi(G)$, тогда число цветов меньше минимального возможного. Следовательно, существуют 2 смежные вершины, цвета которых одинаковы. Что противоречит логике алгоритма. $\square$

Следствие 2.1. $\chi(G) \leq \Delta(G) + 1$, где $\Delta(G)$ - максимальная степень вершины.

Доказательство. Промоделируем жадный алгоритм: 1. Рассмотрим произвольную вершину v_i . 2. К моменту её окрашивания уже раскрашены некоторые из её соседей. Общее число соседей у v_i не превышает $\deg(v_i) \leq \Delta(G)$. 3. Следовательно, количество различных цветов смежных вершин $\leq \Delta(G)$ 4. Чтобы гарантированно присвоить v_i цвет, отличный от всех цветов её соседей, достаточно использовать один из $\Delta(G) + 1$ цветов.

$$\chi(G) \leq \chi_g(G) \leq \Delta(G) + 1$$

$\square$

Соответственно, если граф поделен на компоненты связности, каждую из них можно рассмотреть отдельно.

Однако, в общем случае равенства $\chi_g(G)$ и $\chi(G)$ не будет. Рассмотрим двудольный граф, котором вершины каждая вершина из $\pi_1 = \{3, 5, 7, \dots, 2n-1\}$ соединена со всеми вершинами $\pi_2 = \{4, 6, 8, \dots, 2n\}$. Вершина 1 смежна со всеми вершинами множества π_2 , а вершина 2 – с π_1 . Тогда при последовательной раскраске мы получаем $n/2$ различных цвета, в то время как оптимальный ответ – 2.

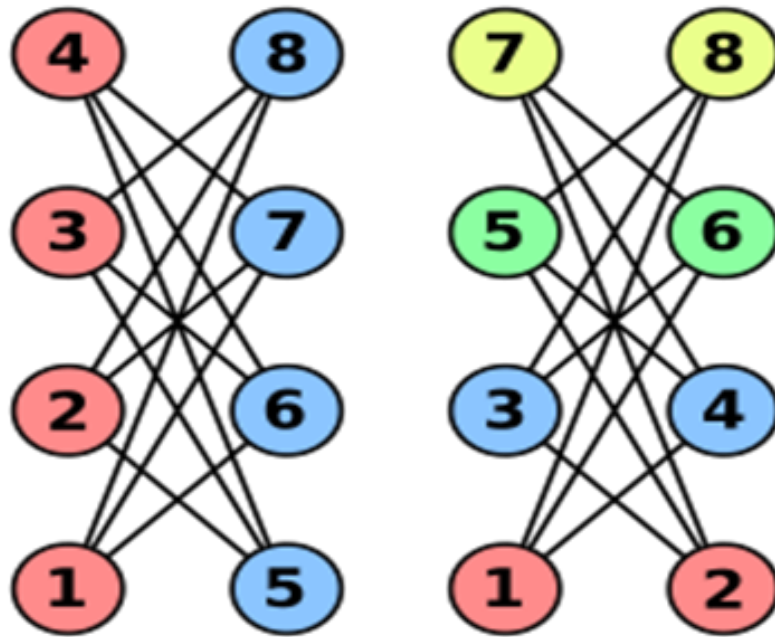


Рис. 2.1: Жадный алгоритм для двудольного графа

Одним из способов разрешения этой проблемы – более удачный выбор последующего элемента.

2.2 DSatur (Degree of Saturation)

Данный алгоритм идейно похож на алгоритм последовательной раскраски, за одним отличием.

В алгоритме DSatur выбор вершины для раскраски определяется эвристически на основе характеристик текущей частичной раскраски графа. Этот выбор основан в первую очередь на степени насыщенности вершин, определяемой следующим образом:

Пусть $c(v) = NULL$ для любой вершины $v \in V$, которая в данный момент не назначена цветовому классу. Для такой вершины v , степень насыщения, обозначаемая как $sat(v)$ — это количество различных цветов, назначенных смежным вершинам. То есть:

$$sat(v) = |\{c(u) : u \in \Gamma(v) \wedge c(u) \neq NULL\}|, \text{ где}$$

$c(v)$ — цвет вершины v (NULL если не раскрашена) $\Gamma(v)$ — окрестность вершины v (множество смежных вершин)

В качестве второго критерия обычно выбирают степень вершины.

Листинг 2.2: Алгоритм DSatur

```

1      vector<int> dsatur(vector<vector<int>>& graph) {
2          int n = graph.size();
3          vector<int> colors(n, -1);
4
5          int vert = 0;
6          int mx = graph[0].size();
7          for (int i = 0; i < n; i++)
8              if (mx < graph[i].size()) {
9                  mx = graph[i].size();
10                 vert = i;
11             }
12
13         vector<int> cnt(n);
14         vector<vector<bool>> saturation(n, vector<bool>(n))
15             ;
16         int colored = 0;
17         while (colored < n) {
18             vector<bool> available(n, 0);
19             for (int i : graph[vert]) {
20                 if (colors[i] != -1)
21                     available[colors[i]] = 1;
22             }
23
24             int min_color = 0;
25             while (available[min_color] == 1)
26                 min_color++;
27
28             colors[vert] = min_color;
29             colored++;

```

```

30         for (int i : graph[vert]) {
31             if (saturation[i][min_color] == 0)
32                 cnt[i]++;
33             saturation[i][min_color] = 1;
34         }
35
36         int mx = -1, next_vertex = -1, deg = -1;
37         for (int i = 0; i < n; i++) {
38             if (colors[i] == -1 && (mx < cnt[i] ||
39                 mx == cnt[i] && (int)graph[i].size() > deg)
40                 ) {
41                 mx = cnt[i];
42                 next_vertex = i;
43                 deg = graph[i].size();
44             }
45
46             vert = next_vertex;
47         }
48         return colors;
49     }

```

Чтобы выбрать следующую вершину, мы проводим 2 процедуры:

1. Пересчет степеней насыщения – число смежных вершин. $O(n)$
2. Перебор всех вершин и выбор оптимальной вершины. $O(n)$

В итоге получаем сложность $O(n^2)$. И если для предыдущего алгоритма сложность $O(n^2)$ была грубой, то для DSatur, она будет вполне точной, т.к. в ходе работы раскрашиваются все n вершин и в качестве перехода гарантированно рассматриваются n возможных вариантов.

В качестве преимущества DSatur над предыдущим алгоритмом, выделим следующие факты.

Теорема 2.1. *Алгоритм DSATUR является точным для циклов и колесных графов.*

Доказательство. Пусть C_n — граф-цикл. Поскольку степень всех вершин в C_n равна 2, первая вершина v , подлежащая раскраске, будет выбрана алгоритмом DSATUR произвольно. На следующих $n-2$ шагах, в соответствии с поведением DSATUR, будет построен путь вершин с чередующимися цветами, расширяющийся от v в обоих направлениях.

В конце этого процесса будет сформирован путь, состоящий из $n - 1$ вершин, и останется единственная вершина u , смежная с обеими концевыми вершинами этого пути.

Если C_n — четный цикл, то $n - 1$ будет нечетным, что означает, что концевые вершины имеют одинаковый цвет. Следовательно, u может быть окрашена в альтернативный цвет.

Если C_n — нечетный цикл, то $n - 1$ будет четным, что означает, что концевые вершины будут иметь разные цвета. Следовательно, для u будет назначен третий цвет.

Для колесных графов W_n применимо аналогичное рассуждение. Предполагая $n \geq 5$, DSATUR изначально раскрасит центральную вершину v_n , поскольку она имеет наивысшую степень. Так как v_n смежна со всеми остальными вершинами в W_n , все оставшиеся вершины $v_1, \dots, v_{n-1}$ теперь будут иметь степень насыщения, равную 1. Затем следует тот же процесс раскраски, что и для графов-циклов C_{n-1} . $\square$

Докажем вспомогательное утверждение:

Теорема 2.2 (о двудольности). *Граф является двудольным тогда и только тогда, когда он не содержит циклов нечетной длины.*

Доказательство. (Необходимость) Допустим в двудольном графе $G = (E, V_1, V_2)$ существует цикл $\pi = (v_1, v_2, \dots, v_s)$. Без ограничения общности, допустим, что $v_1 \in V_1$. Тогда $v_i \in V_1$ тогда и только тогда, когда i нечетно. И наоборот $v_i \in V_2$ только в том случае, когда i четно. Тогда чтобы из v_s попасть в v_1 , s должно быть четным.

(Достаточность) Теперь предположим, что известно, что в G нет циклов нечетной длины. Выберем произвольную вершину v в графе и определим множество V_1 как множество вершин, для которых кратчайший путь от каждой вершины из V_1 до v имеет нечетную длину, а V_2 — как множество вершин, где кратчайший путь от каждой вершины из V_2 до v имеет четную длину. Заметим теперь, что нет ребер, соединяющих вершины одного и того же множества V_i , поскольку в противном случае G содержал бы цикл нечетной длины. Следовательно, G является двудольным. $\square$

Теорема 2.3 (Брелаз (1979)). *Алгоритм DSatur является точным для двудольных графов.*

Доказательство. Пусть G — связный двудольный граф с $n \geq 3$. Если граф G не является связным, достаточно рассмотреть каждую его компоненту связности отдельно.

Предположим, что некоторая вершина x имеет степень насыщения, равную двум; в этом случае у неё есть два соседа, окрашенных в разные цвета, и мы можем построить два пути. Поскольку граф G связан, мы находим общую для этих путей вершину y , следовательно, образуется цикл. Либо этот цикл будет четным, и тогда два соседа вершины x должны иметь одинаковый цвет, либо граф G не является двудольным. Следовательно, алгоритм DSatur является точным для двудольных графов. $\square$

2.3 Эксперименты

Проведем сравнительный анализ алгоритма последовательной раскраски и DSatur.

Критерии сравнения:

1. Качество решения: Определяется количеством цветов k , используемых в финальной раскраске. Чем меньше k , тем ближе решение к истинному (хроматическому числу $\chi(G)$).

2. Время выполнения: Измеряется процессорное время, затраченное на выполнение каждого алгоритма. Позволяет оценить вычислительную производительность.

Затраты памяти в рассмотрение брать не будем, так как с этим problem возникнуть не должно.

Тестовые данные: Для экспериментов будут использованы случайные графы с заданным числом ребер и фиксированным числом вершин (500).

Процедура: Каждый алгоритм будет запущен на тестовых графах с определенным числом случайно сгенерированных ребер 20 раз. В качестве результата братья будет среднее арифметическое.

Планируется, что DSatur будет работать более точно, путем не асимптотически больших временных затрат.

Поскольку точной верхней асимптотической оценкой работы жадных алгоритмов можно назвать $O(n^2 + m)$ для DSatur и $O(n + m)$ для Greedy.

Так как n фиксировано, то можно предположить, что при увеличении числа ребер время работы будет меняться линейно, то есть $O(m)$.

Приведем результаты экспериментов для $n = 500$. Число ребер перебираем от 0 до $n * (n - 1)/2$ с шагом 1000. Ребра добавляем случайным образом.

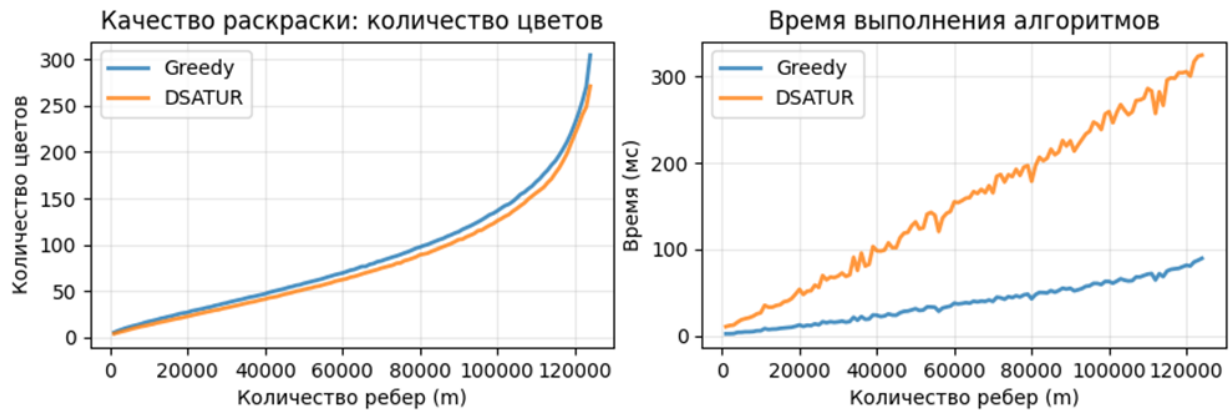


Рис. 2.2

Ожидаемо, число цветов для DSatur меньше чем для Greedy. При этом, визуально время работы похоже на линейное, но с разными коэффициентом наклона. Приведем отношение двух признаков.



Рис. 2.3

Можно обратить внимание, что отношения времени работы уменьшается, то есть время работы Greedy увеличивается относительно DSatur с увеличением числа ребер.

Соотношение числа цветов меняется, в силу того, что общий ответ становится больше. Для большей наглядности приведем разность ответов двух решений.

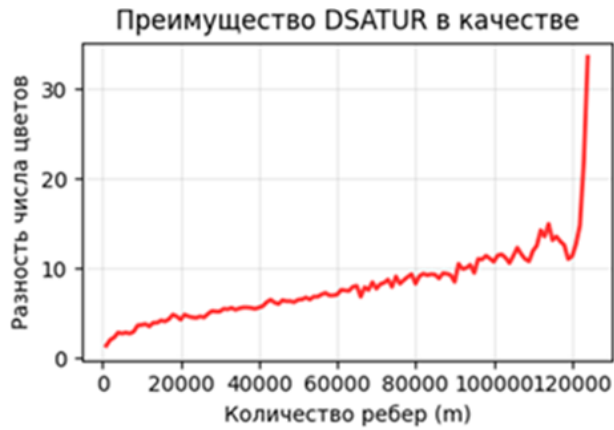


Рис. 2.4

Далее проверим предположение о линейной зависимости времени работы от числа ребер. Для этого поделим время на текущее значения количества ребер, чтобы отношение имело более приемлемый вид домножим это значение на 10^4 .



Рис. 2.5

Отсюда видно что отношение для Greedy и DSatur стремится к постоянной величине: ≈ 7 и ≈ 25 соответственно. Из чего следует вывод о линейной зависимости времени работы DSatur и Greedy от числа ребер.

Далее, проведем следующий эксперимент. Для этого введем метрику плотности графа. Плотность графа – это числовая характеристика, которая показывает, насколько граф близок к полному графу. Где 1 – полный граф.

Тестовые данные: Для экспериментов будут использованы случайные графы с заданным числом вершин и фиксированной плотностью ($1/4$).

Процедура: Каждый алгоритм будет запущен тестовых графах с фиксированной плотностью и случайно сгенерированными ребрами 20 раз.

В качестве результата братья будет среднее арифметическое. Для разных алгоритмов графы будут одни и те же.

Примем в качестве $m = \frac{n^2}{4}$

$$O(n^2 + m) = O(n^2 + \frac{n^2}{4}) = O(n^2)$$

$$O(n + m) = O(n + \frac{n^2}{4}) = O(n^2)$$

Ожидается, что время работы будет квадратично меняться при изменении числа вершин.

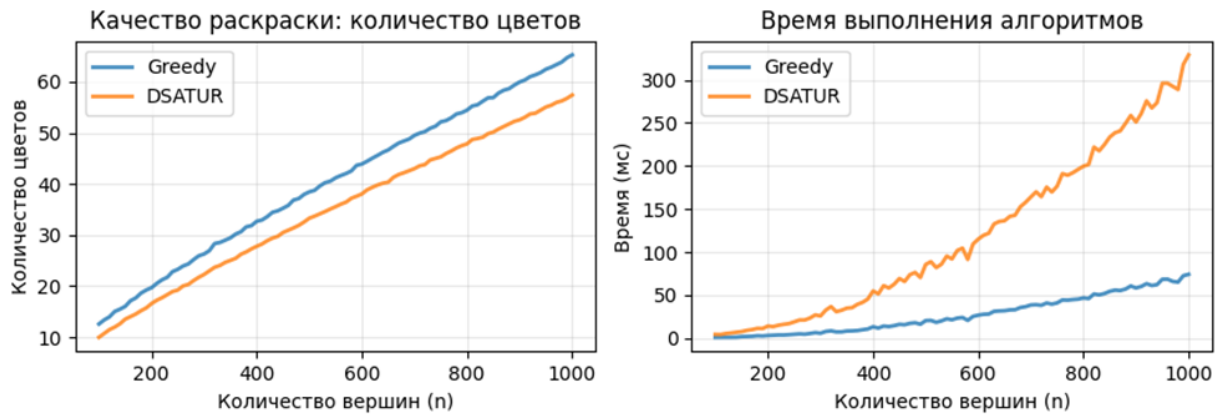


Рис. 2.6

Также как и в предыдущем эксперименте, DSatur показывает лучшее качество решения, путем более длительной работы. Также можно сделать вывод о квадратичной зависимости от числа вершин. Более детально этот вопрос затронем далее. Пока что рассмотрим отношения соответствующих критериев у двух алгоритмов.

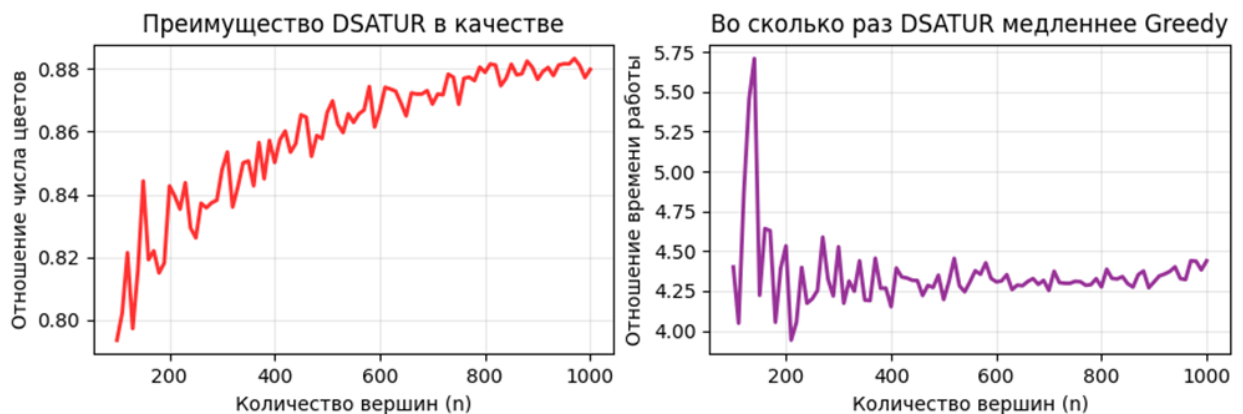


Рис. 2.7



Рис. 2.8

Отношение числа цветов и времени работы находится примерно на том же уровне, что и в прошлом эксперименте. Разница в том, что отношение времени работы теперь приближено к постоянному. Интересной также является разность качества решений, которая отдаленно напоминает график линейной функции.

Далее рассмотрим вопрос квадратичной зависимости от числа вершин. Для этого поделим время работы на n^2 . Чтобы ответ находился в приемлемых рамках домножим его на 10^4 .

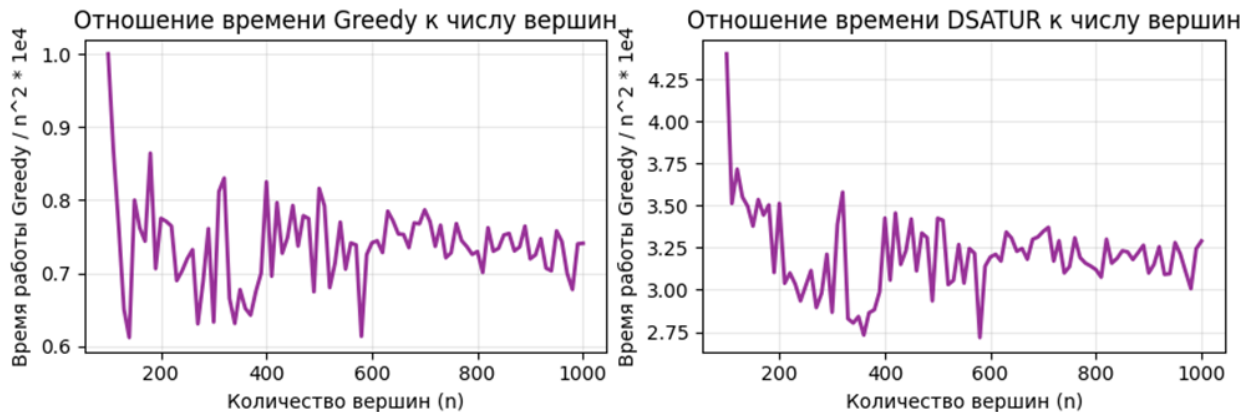


Рис. 2.9

Отсюда видно, что при увеличении числа вершин отношение устремляется к какой-то постоянной величине. Таким образом, мы смогли подтвердить предположение о квадратичном виде графика.

Вывод: эксперименты показали, что путем увеличения времени работы в мы получили лучшее решение, которое показывает более оптимальный результат. Также мы проверили линейную зависимость для 1-го эксперимента и квадратичную зависимость для 2-го.

3 Конструктивные алгоритмы

3.1 RLF

Алгоритм Recursive Largest First (RLF), предложенный Фрэнком Лем в 1979 году, представляет собой эффективный эвристический метод решения задачи раскраски графа. RLF строит раскраску путём итеративного формирования независимых множеств, каждое из которых получает свой цвет.

Основная идея алгоритма заключается в следующем: на каждом шаге строится максимальное (по включению) независимое множество вершин, которое раскрашивается в новый цвет. Затем эти вершины удаляются из графа, и процесс повторяется для оставшегося подграфа. Ключевым моментом является стратегия выбора вершин при построении независимого множества: приоритет отдаётся вершинам с наибольшей степенью в остаточном графе, что позволяет минимизировать количество используемых цветов.

Алгоритм RLF строит раскраску последовательно по цветам. Для текущего цвета c формируется множество $S_c \subset V$ — независимое множество вершин, получающих цвет c . Процесс построения S_c подчиняется следующим правилам:

1. Первая вершина в S_c выбирается как вершина с максимальной степенью в текущем остаточном графе.
2. Каждая следующая вершина выбирается из множества вершин, не смежных ни с одной из уже выбранных (то есть потенциально возможных для добавления в S_c).
3. Выбор осуществляется по критерию максимизации числа вершин, которые станут недоступными для добавления после включения данной

вершины (что эквивалентно выбору вершины с максимальным числом соседей среди уже выбранных).

Algorithm 1: Recursive Largest First (RLF)

Вход : Граф $G = (V, E)$, $|V| = n$
Выход: Раскраска $\phi : V \rightarrow \mathbb{N}$

```

1   $color \leftarrow 1$ ;
2   $U \leftarrow V$ ;
3  Для всех  $v \in V$ :  $\phi(v) \leftarrow 0$ ;
4  while  $U \neq \emptyset$  do
5      |   Выбрать  $v \in U$  с максимальной степенью в  $G[U]$ ;
6      |    $S \leftarrow \{v\}$ ;
7      |    $A \leftarrow N(v) \cap U$ ;
8      |    $W \leftarrow U \setminus (S \cup A)$ ;
9      |   while  $W \neq \emptyset$  do
10     |       |   Для каждой вершины  $w \in W$  вычислить:
10     |       |       
$$d_A(w) = |N(w) \cap A|$$

10     |       |       Выбрать  $w^* = \arg \max_{w \in W} d_A(w)$ ;
11     |       |        $S \leftarrow S \cup \{w^*\}$ ;
12     |       |        $A \leftarrow A \cup (N(w^*) \cap W)$ ;
13     |       |        $W \leftarrow W \setminus (\{w^*\} \cup N(w^*))$ ;
14     |   end while
15     |   Для всех  $v \in S$ :  $\phi(v) \leftarrow color$ ;
16     |    $U \leftarrow U \setminus S$ ;
17     |    $color \leftarrow color + 1$ ;
18 end while
19 return  $\phi$ ;

```

U — множество ещё не раскрашенных вершин

$G[U]$ — это подграф, образованный только нераскрашенными вершинами.

S — строящийся цветовой класс (независимое множество)

A — это соседи v среди нераскрашенных вершин.

W — кандидаты на добавление в S

Оценим сложность реализации алгоритма, с использованием матрицы смежности:

Утверждение 3.1 (Временная сложность алгоритма RLF). *Алгоритм*

RLF требует $O(n^3)$ времени и $O(n^2)$ памяти для графа из n вершин, m ребер.

Доказательство. Внешний цикл выполняется $O(n)$ раз (предполагаемое число цветов). Выбор начальной вершины с максимальной степенью в $G[U]$ требует $O(n^2)$ операций при матричном представлении графа. Построение множеств A и W выполняется за $O(n)$.

Основные затраты приходятся на внутренний цикл. На каждом его шаге для всех вершин из W вычисляется $d_A(w) = |N(w) \cap A|$, что требует $O(|W| \cdot |A|) = O(n^2)$ операций. Число шагов внутреннего цикла равно размеру цветового класса $|S|$, а сумма $|S|$ по всем итерациям составляет n . Следовательно, суммарная стоимость выбора w^* равна $O(n^3)$.

Операции обновления S , A , W и присваивания цветов дают в сумме $O(n^2)$ и не влияют на асимптотику. Таким образом, алгоритм RLF имеет временную сложность $O(n^3)$. $\square$

При этом стоит отметить, что оценка довольно грубая, а время работы алгоритма зависит, в том числе, и от хроматического числа (которое мы сверху оценили как n).

Теорема 3.1. *Алгоритм RLF является точным для двудольных графов.*

Доказательство. Пусть G — связный двудольный граф с $n \geq 3$ и долями V_1 и V_2 . Если G несвязен, достаточно рассмотреть каждую компоненту отдельно.

Без ограничения общности предположим, что вершина $v \in V_1$ имеет наибольшую степень и, следовательно, первой выбирается в цветовой класс S (ей присваивается цвет 0). Все соседи v , то есть $\Gamma(v) \subseteq V_2$, попадают в множество A . Теперь достаточно показать, что следующие $|V_1| - 1$ вершин, добавленных в S , также будут из V_1 . Это действительно так, поскольку на каждом из следующих $|V_1| - 1$ шагов из множества W будут выбираться нераскрашенные вершины, имеющие наибольшее число соседей в A . Так как нераскрашенные вершины из $V_2 \setminus A$ не смежны с вершинами из A , выбираться будут только вершины из V_1 . Это продолжается до тех пор, пока все вершины из V_1 не получают цвет 0. В этот момент подграф, индуцированный V_2 , не содержит рёбер, что позволяет RLF раскрасить все оставшиеся вершины цветом 1. $\square$

Теорема 3.2. *Алгоритм RLF является точным для циклов и колёс.*

Доказательство. Чётные циклы являются 2-раскрашиваемыми и, следовательно, охватываются теоремой 2.11. Однако для удобства рассмотрим как чётные, так и нечётные циклы. Пусть C_n — цикл с вершинами $V = \{v_1, \dots, v_n\}$ и рёбрами $E = \{\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}, \{v_n, v_1\}\}$. Для определённости будем считать, что при равенстве значений $d_A(w)$ RLF выбирает вершину с наименьшим индексом (вместо произвольного выбора). Нетрудно видеть, что теорема верна и без этого ограничения.

Степень всех вершин в C_n равна 2, поэтому первой будет выбрана v_1 (ей присвоен цвет 0). Соседи v_2 и v_n попадают в A . Согласно эвристике RLF, следующей будет выбрана v_3 , затем v_5 и так далее. В конце этого процесса цветовой класс $S_0 = \{v_1, v_3, \dots, v_{n-1}\}$ для чётного n и $S_0 = \{v_1, v_3, \dots, v_{n-2}\}$ для нечётного n . В чётном случае остаётся подграф с вершинами $\{v_2, v_4, \dots, v_n\}$ без рёбер, и RLF присваивает им всем цвет 1. В нечётном случае остаются нераскрашенные вершины $\{v_2, v_4, \dots, v_{n-1}, v_n\}$ с единственным ребром $\{v_{n-1}, v_n\}$. Следуя эвристике RLF, все вершины с чётными индексами получают цвет 1, а v_n — цвет 2.

Для колёс W_n применимы аналогичные рассуждения. При $n \geq 5$ центральная вершина v_n имеет наибольшую степень и получает цвет 0. Поскольку v_n смежна со всеми остальными вершинами, никакие другие вершины не могут быть добавлены в S_0 , и алгоритм переходит к цвету 1. Оставшиеся нераскрашенные вершины образуют цикл C_{n-1} , после чего процесс раскраски продолжается как описано выше. $\square$

3.2 Эксперименты

Для начала сравним текущий алгоритм с теми, что были в главе 2 (жадные алгоритмы).

Для этого проведем серии экспериментов, аналогичные тем, что были во 2-ой главе.

1) Изменение числа ребер, при фиксированном числе вершин.

Тестовые данные: Для экспериментов будут использованы случайные графы с заданным числом ребер и фиксированным числом вершин

(500).

Процедура: Каждый алгоритм будет запущен на тестовых графах с определенным числом случайно сгенерированных ребер 20 раз. В качестве результата братья будет среднее арифметическое.

Гипотеза: Ожидается, что RLF будет находить более оптимальную раскраску, затрачивая на это (асимптотически) больше времени.

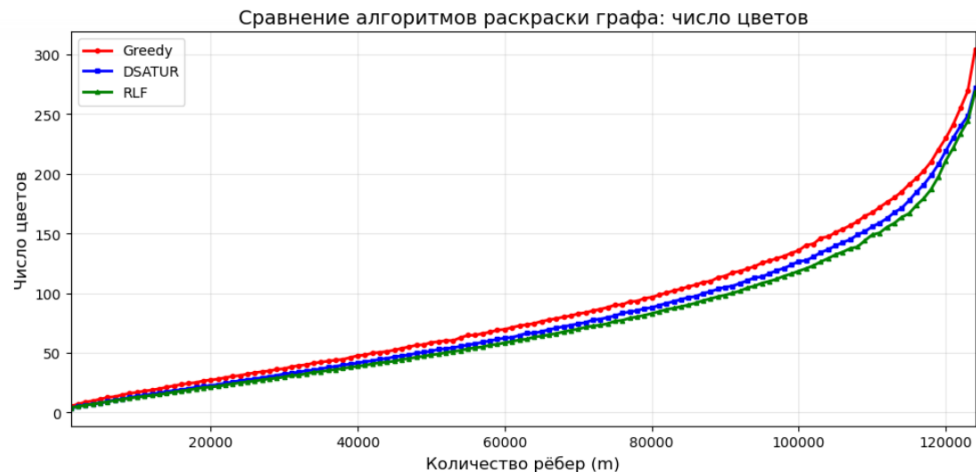


Рис. 3.1

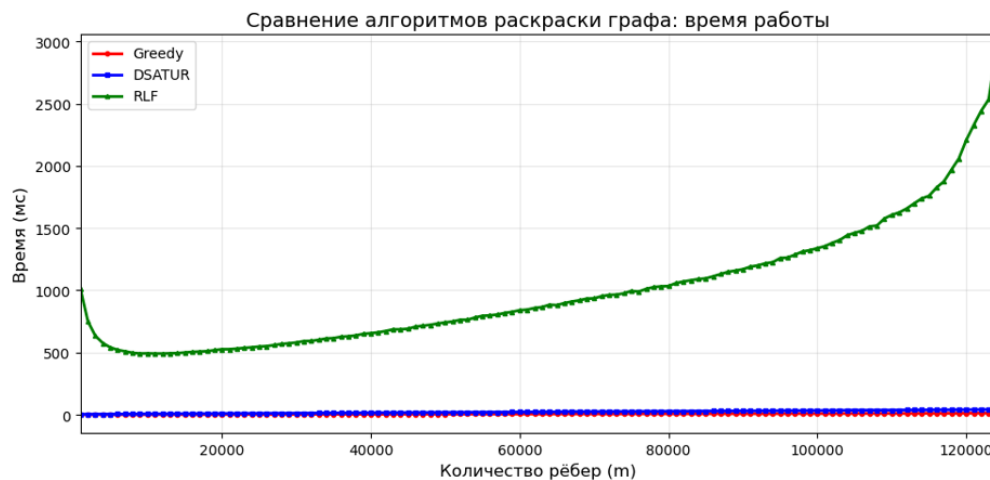


Рис. 3.2

Откуда видим, что RLF дает более качественную раскраску, чем жадные алгоритмы, но при этом использует сильно больше времени.

При этом, если время предыдущих двух алгоритмов росло пропорционально числу ребер, здесь такой однозначности нет, так как время работы зависит не только от числа ребер, но и от характеристического числа, которое убывает с ростом m .

Далее более детально сравним RLF и DSatur

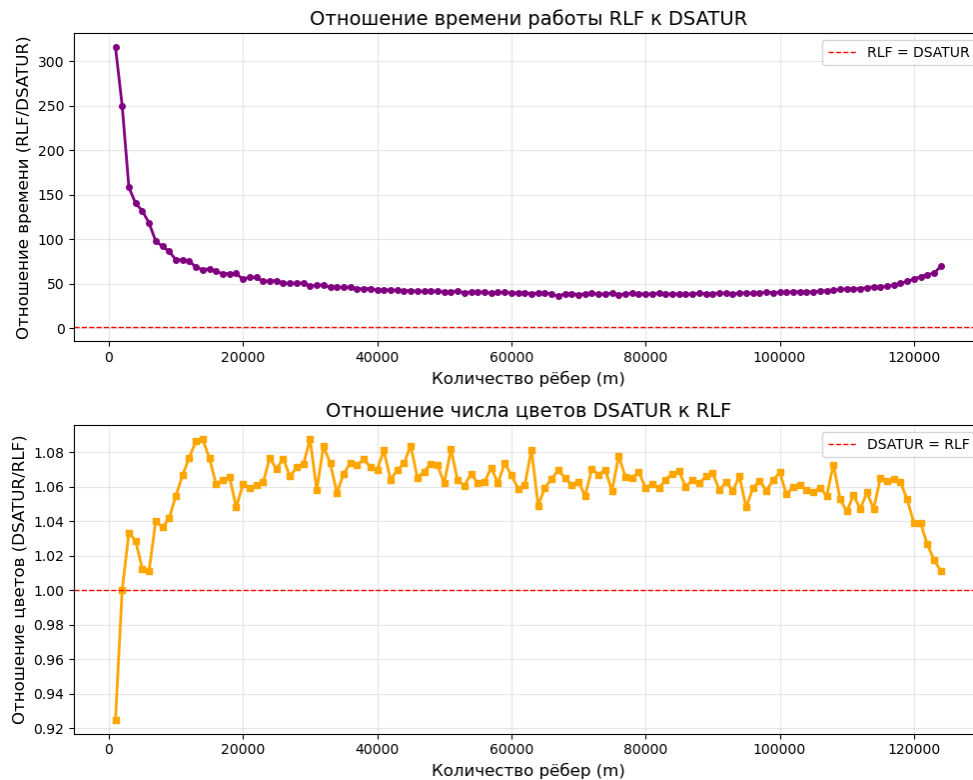


Рис. 3.3

Красным пунктиром выделено отношение равное единице. Экспериментально получилось, что среднее отношение времени работы ≈ 53 , а среднее отношение числа цветов ≈ 1.06 .

При этом любопытно, что отношение времени работы на достаточно большом участке представляет из себя постоянную величину. Однако при более детальном рассмотрении оказывается, что это не совсем так. Сначала соотношение падает, потом возрастает.

2) Изменение числа вершин при фиксированной плотности графа

Тестовые данные: Для экспериментов будут использованы случайные графы с заданным числом вершин и фиксированной плотностью $= 1/4$.

Процедура: Каждый алгоритм будет запущен на тестовых графах с определенным числом случайно сгенерированных ребер 20 раз. В качестве результата братья будет среднее арифметическое.

Гипотеза: Ожидается, что RLF будет находить более оптимальную раскраску, затрачивая на это (асимптотически) больше времени.

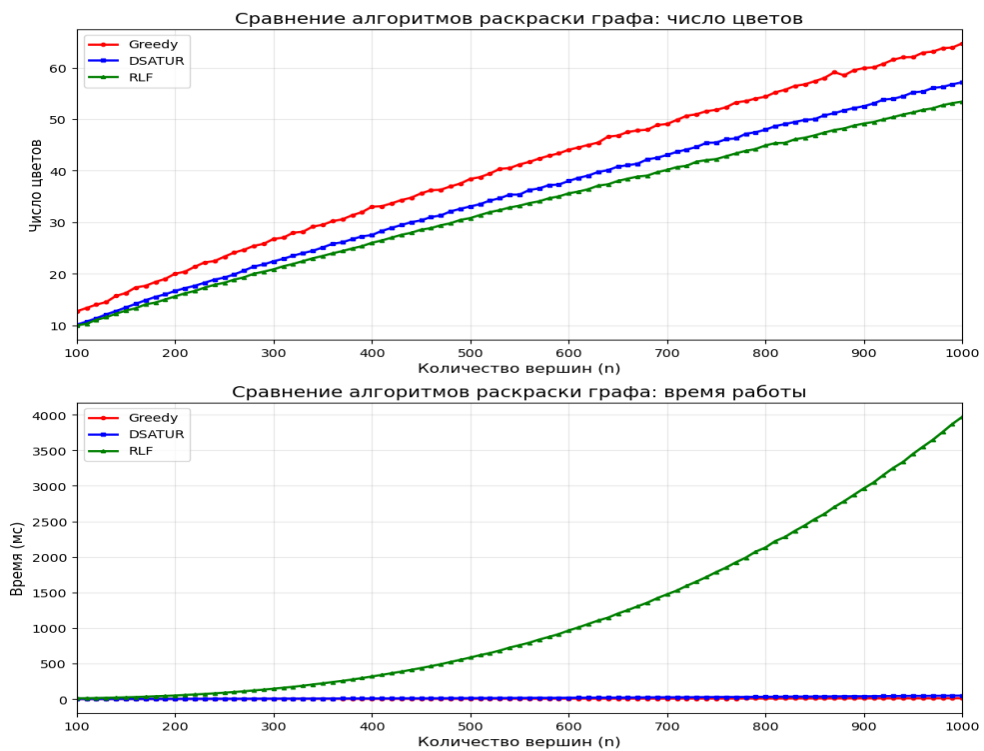


Рис. 3.4

Так как сложность алгоритма $O(n^3)$, то время работы монотонно растёт с увеличением числа вершин.

Перейдем к более детальному сравнению RLF и DSatur.

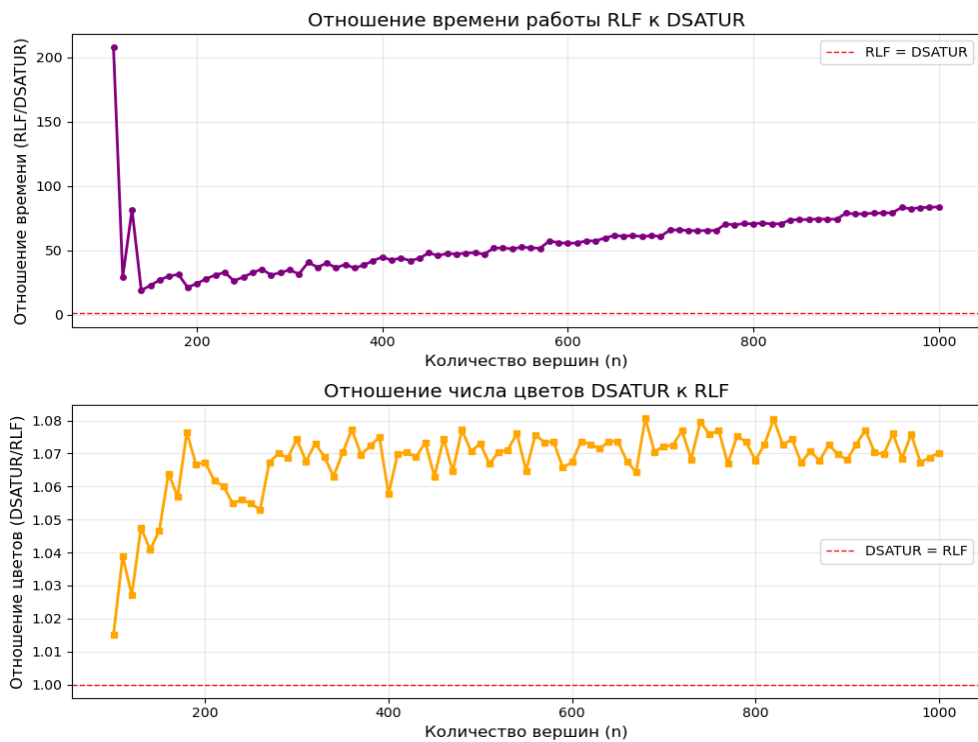


Рис. 3.5

Отношение времени работы ≈ 50 .

Отношение количества цветов ≈ 1.07

Также отметим, что график отношения времени работы напоминает линейную функцию. Обосновать это аналитически можно следующим образом:

$$O(n^3)/O(n^2) = C * n, C = const$$

3) Экспериментально проверим сложность алгоритма.

Мы предполагаем, что RLF имеет временную сложность $O(n^3)$ и пространственную сложность $O(n^2)$.

Если пространственная сложность вопросов не вызывает, то временную имеет смысл проверить экспериментально.

Из 1-го эксперимента мы знаем, что наибольшее время работы RLF при фиксированном числе вершин соответствует полному графу.

Исходя из этого проведем следующий эксперимент: для полных графов будем считать время их работы при заданном n и $m = \binom{n}{2}$, и поделим его [время работы] на предположительную сложность.

Тогда если оценка окажется точной, то соотношение будет находиться в ε -окрестности какой-то константы. Если оценка завышена, то значение будет монотонно убывать. Если отношение возрастает – оценка неверна.

Тестовые данные: Для экспериментов будут использованы случайные графы с заданным числом вершин и фиксированной плотностью = 1.

Процедура: RLF будет запущен на тестовых графах с определенным числом случайно сгенерированных ребер 20 раз. В качестве результата братья будет среднее арифметическое.

Гипотеза: Алгоритм имеет сложность $O(n^3)$, но оценка завышена.

Чтобы отношение было больше единицы домножим его на 10^5 .

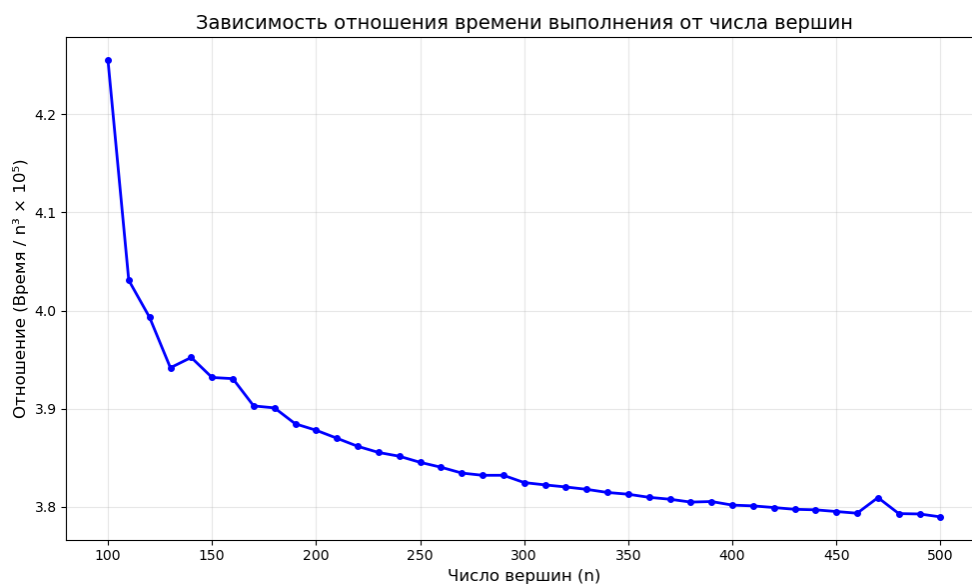


Рис. 3.6

Как и ожидалось, алгоритм имеет сложность $O(n^3)$, при этом даже для полного графа оценка n^3 операций оказалась завышенной.

4 Точные алгоритмы

Одним из способов точного решения NP-полных задач является метод перебора. Так как жадные алгоритмы не всегда дают оптимальный результат, в некоторых случаях можно просто перебрать все возможные варианты раскраски. Но у такого метода есть очевидная издержка в виде экспоненциальной верхней асимптотической оценки.

4.1 Перебор вершин

Для заданного графа $G = (V, E)$ вершины сначала упорядочиваются определенным образом, так что вершина v_i ($1 \leq i \leq n$) соответствует i -й вершине в этом порядке. Затем алгоритм совершает серию недетерминированных переходов.

1. Раскрашиваем первую вершину в один из возможных k цветов. Без ограничения общности, пусть это будет 0.

2. Заранее определенным образом выбираем следующую вершину и рекурсивно выбираем для нее один из возможных цветов, не конфликтующих с уже окрашенными соседями. Продолжаем выполнение алгоритма для всех узлов дерева решений.

3. Если все вершины покрашены, фиксируем раскраску и количество использованных цветов. Алгоритм завершается, когда все ветви решения вычислены. Оптимальным решением считается раскраска с наименьшим количеством цветов среди всех зафиксированных.

Приведем наивную реализацию на C++:

Листинг 4.1: Наивный алгоритм перебора вершин

```
1 int find_chromatic_number(int v, vector<vector<int>>&  
edges, vector<int> solution) {  
2     if (v == n) {
```

```

3         int cnt = 0;
4         vector<int> cnt_col(n);
5         for (int i = 0; i < n; i++) {
6             cnt_col[solution[i]]++;
7             if (cnt_col[solution[i]] == 1)
8                 cnt++;
9         }
10        best = min(best, cnt);
11        return cnt;
12    }
13
14    int min_ans = n;
15
16    if (v == 0) {
17        solution[v] = 0;
18        min_ans = find_chromatic_number(v + 1, edges,
19                                         solution);
19    }
20    else {
21        for (int color = 0; color < n; color++) {
22            solution[v] = color;
23            bool correct = 1;
24            for (int i = 0; i < edges[v].size(); i++) {
25                if (solution[edges[v][i]] == color)
26                    correct = 0;
27            }
28            if (correct) {
29                int cur_ans = find_chromatic_number(v +
30                                                     1, edges, solution);
31                min_ans = min(min_ans, cur_ans);
32            }
33        }
34        return min_ans;
35    }

```

В крайнем случае, решение будет перебирать все n^n наборов, на каждом шаге перебирая n цветов и n смежных вершин. Таким образом, получаем оценку $O(n^n * n^2)$

Такая реализация, называемая наивной, представляет из себя переписывание вышеизложенных шагов на C++. Однако она имеет простор

для добавления оптимизаций и эвристик.

4.1.1 Отсечения по числу цветов

При наивной реализации возникает проблема с тем, что мы в узлах дерева решений рассматриваем варианты, которые заведомо не дадут нам решений, лучше имеющихся. В связи с этим имеет смысл их «отсекать».

Оптимизация 1: Вместо того, чтобы рассматривать всевозможные размещения цветов можно фиксировать уже имеющиеся ответы и по их значениям, и если в текущей раскраске число цветов больше или равно одному из ответов, полученных ранее, выполнять этот узел дерева рекурсии смысла не имеет.

Также имеет смысл делать отсечения и по максимальному значению цвета. Поскольку, если в оптимальной раскраске χ цветов, то найдется и такая, что все цвета $< \chi$ (если индексация начинается с нуля).

Оптимизация 2: Можно получить верхнюю оценку для числа цветов используя, изложенный в главе 2 теоретический материал. Т.е. в качестве верхней оценки можно взять результат работы DSatur или любого другого полиномиального алгоритма.

4.1.2 Выбор следующей вершины

Для более удачного выбора последующей вершины можно использовать, какой-нибудь эвристический метод, рассчитывая тем самым получить лучшее отсечение. В частности, можно использовать принцип «Степеней насыщения», описанный ранее.

4.1.3 Рационализация

Известны различные приемы сокращения перебора при использовании описанной стратегии исчерпывающего поиска. Один из них основан на следующем наблюдении:

Допустим, имеются две вершины a и b такие, что каждая вершина, отличная от b и смежная с вершиной a , смежна и с вершиной b . Иначе говоря, $V(a) - \{b\} \subseteq V(b)$. Будем говорить в этом случае, что вершина b

поглощает вершину a . Если при этом вершины a и b смежны, то скажем, что вершина b смежно поглощает вершину a . Вершину b в этом случае назовем смежно поглощающей.

Утверждение 4.1. *Если вершина a является несмежно поглощаемой в графе, то $\chi(G - a) = \chi(G)$.*

Доказательство. Допустим, что нашли оптимальную раскраску для $G - a$. Тогда, если вершина a несмежно поглощается вершиной b , то её можно раскрасить в тот же цвет в графе G . $\square$

Эту процедуру довольно просто реализовать используя матрицу смежности. То есть рассмотреть всевозможные пары несмежных вершин $O(n^2)$ и проверить, поглощает ли одна другую $O(n)$. Итоговая сложность – $O(n^3)$.

Листинг 4.2: Рационализация

```

1      vector<vector<int>> rationalization(int n, vector<
2          vector<int>>& matrix) {
3          vector<bool> banned(n);
4
5          for (int i = 0; i < n; i++) {
6              for (int j = i + 1; j < n; j++) {
7                  if (matrix[i][j] == 0) {
8                      bool absort = 1;
9                      for (int k = 0; k < n; k++)
10                         if (matrix[i][k] < matrix[j][k])
11                            absort = 0;
12                     if (absort) {
13                         banned[j] = 1;
14                         continue;
15                     }
16                     absort = 1;
17                     for (int k = 0; k < n; k++)
18                         if (matrix[j][k] < matrix[i][k])
19                            absort = 0;
20                     if (absort) {
21                         banned[i] = 1;
22                         continue;
23                     }
24                 }
25             }
26         }
27     }

```

```

25     }
26
27     vector<vector<int>> edg(n);
28     for (int i = 0; i < n; i++)
29     for (int j = i + 1; j < n; j++) {
30         if (banned[i] == 0 && banned[j] == 0 && matrix[
31             i][j] == 1) {
32             edg[i].push_back(j);
33             edg[j].push_back(i);
34         }
35     }
36     return edg;

```

На вход получаем матрицу смежности – возвращаем список смежно-
сти.

4.1.4 Реализация

Листинг 4.3: Реализация перебора

```

1     best = INF
2
3     int vertex_backtrack(int v, vector<vector<int>>& edges,
4         vector<int> solution, int large_color) {
5         if (large_color >= best - 1)
6             return INF;
7
8         if (v == n) {
9             int cnt = 0;
10            vector<int> cnt_col(n);
11            for (int i = 0; i < n; i++) {
12                cnt_col[solution[i]]++;
13                if (cnt_col[solution[i]] == 1)
14                    cnt++;
15            }
16            best = min(best, cnt);
17            return cnt;
18        }
19
20        int min_ans = n;

```

```

20
21     if (v == 0) {
22         solution[v] = 0;
23         int next_vertex;
24         //choose next vertex
25         min_ans = vertex_backtrack(next_vertex, edges,
26                                     solution, 0);
27     }
28     else {
29         for (int color = 0; color < best - 1; color++)
30         {
31             solution[v] = color;
32             bool correct = 1;
33             for (int i = 0; i < edges[v].size(); i++) {
34                 if (solution[edges[v][i]] == color)
35                     correct = 0;
36             }
37             if (correct) {
38                 int next_vertex;
39                 // choose next vertex
40                 int cur_ans = vertex_backtrack(
41                     next_vertex, edges, solution, max(
42                         color, large_color));
43                 min_ans = min(min_ans, cur_ans);
44             }
45         }
46     }
47     return min_ans;
48 }

```

В качестве следующей вершины можно брать ранее не просмотренную вершину, в соответствие с любым правилом, будь то $v + 1$ или степень насыщения. Обе реализации находятся в Приложении.

4.2 Метод Зыкова

Следующий метод включает в себя процедуру стягивания вершин.

Опр. Стягивание вершин – это операция, которая из графа G делает граф $G - uv$, отождествляя вершины uv , заменяя их на одну вершину w , которая смежна тем вершинам, которые были смежны для u и v .

Пусть в графе есть 2 несмежные вершины x, y . Тогда рассмотрим 2 графа: G_1 – полученный путем стягивания вершин x, y G_2 – полученный путем добавление ребра между x, y

Утверждение 4.2. $\chi(G) = \min\{\chi(G_1), \chi(G_2)\}$

Доказательство. Допустим, мы имеем оптимальную раскраску графа G . Тогда возможны 2 варианта: 1. x, y имеют одинаковый цвет. Тогда $\chi(G) = \chi(G_1)$, а $\chi(G) \leq \chi(G_2)$ 2. x, y имеют разный цвет. Тогда $\chi(G) = \chi(G_2)$, а $\chi(G) \leq \chi(G_1)$

Поскольку для оптимальной раскраски всегда выполняется один из двух вариантов, получаем $\chi(G) = \min\{\chi(G_1), \chi(G_2)\}$ $\square$

Это рекуррентное соотношение строит бинарное дерево, называемое деревом Зыкова, в котором листовыми узлами являются клики, а хроматическое число графа определяется по наименьшей клике этого дерева.

Таким образом, мы получаем возможность рекурсивного нахождения раскраски графа в минимальное количество цветов.

Эту рекурсию можно рассматривать как последовательность 2-ух операций: стягивание вершин и добавления ребер. Конечным результатом будет полный граф, как его раскрашивать мы знаем.

Утверждение 4.3. Алгоритм, основанный на деревьях Зыкова, имеет временную сложность $O(2^{n^2} * \text{poly}(n, m))$ и пространственную сложность $O(n^4)$, если на вход подается граф с n вершинами и m ребрами.

Доказательство. Высота дерева Зыкова не превышает $\bar{m}$ — количества отсутствующих ребер, необходимых для полноты графа. Каждый уровень i дерева содержит 2^i узлов, поэтому размер дерева равен $\sum_{i=0}^{\bar{m}} 2^i = 2^{\bar{m}+1} - 1 \leq 2^{\frac{n(n+1)}{2}} - 1 = O(2^{n^2})$

Объем памяти в данном случае равен $O(n^4)$, т.к. при любом методе хранения графа будет затрачено не более $O(n^2)$ памяти, а высота дерева будет не более $O(n^2)$ узлов. $\square$

Листинг 4.4: Наивный метод Зыкова

```

1  int zykov_method(vector<vector<bool>>& g) {
2      int n = g.size();

```

```

3
4     pair<int, int> res = find_non_adj(g);
5     int u = res.first;
6     int v = res.second;
7
8     if (u == -1 && v == -1) {
9         return n;
10    }
11
12    vector<vector<bool>> g1 = add_edge(g, u, v);
13    vector<vector<bool>> g2 = contract_vertices(g, u, v
14        );
15
16    int chi1 = zikov_method(g1);
17    int chi2 = zikov_method(g2);
18
19    return min(chi1, chi2);
    }

```

Реализация функций `find_non_adj`, `add_egde` и `contract_vertices` находится в приложении.

Приложение

Алгоритм 1: Последовательная раскраска (Greedy)

Листинг 4.5: Алгоритм последовательной раскраски

```
1      vector<int> GreedySequential(vector<vector<int>>& graph)
2      {
3          int n = graph.size();
4          vector<int> colors(n, -1);
5
6          for (int vertex = 0; vertex < n; vertex++) {
7              vector<bool> available(n, 0);
8              for (int j : graph[vertex]) {
9                  if (colors[j] != -1)
10                     available[colors[j]] = 1;
11              }
12
13              int min_color = 0;
14              while (available[min_color] == 1)
15                 min_color++;
16
17              colors[vertex] = min_color;
18          }
19      return colors;
20  }
```

Алгоритм 2: DSatur (Degree of Saturation)

Листинг 4.6: Алгоритм DSatur

```
1      vector<int> dsatur(vector<vector<int>>& graph) {
2          int n = graph.size();
```

```

3      vector<int> colors(n, -1);
4      vector<set<int>> sat(n);
5
6      int vert = 0, mx = graph[0].size();
7      for (int i = 0; i < n; i++)
8      if (mx < graph[i].size()) {
9          mx = graph[i].size();
10         vert = i;
11     }
12
13     queue<int> q;
14     q.push(vert);
15     while (!q.empty()) {
16         int v = q.front();
17         q.pop();
18
19         vector<bool> available(n, 0);
20         for (int i : graph[v]) {
21             if (colors[i] != -1)
22                 available[colors[i]] = 1;
23         }
24
25         int min_color = 0;
26         while (available[min_color] == 1)
27             min_color++;
28
29         colors[v] = min_color;
30
31         for (int i : graph[v]) {
32             sat[i].insert(min_color);
33         }
34
35         int mx = -1, next_vertex = -1, deg = -1;
36         for (int i = 0; i < n; i++) {
37             if (colors[i] == -1 && (mx < (int)sat[i].
38                 size() ||
39                 mx == (int)sat[i].size() && (int)graph[i].
40                 size() > deg)) {
41                 mx = sat[i].size();
42                 next_vertex = i;
43                 deg = graph[i].size();

```

```

42         }
43     }
44
45     if (next_vertex != -1)
46         q.push(next_vertex);
47 }
48 return colors;
49 }

```

Алгоритм 3: Рекурсивный перебор

Листинг 4.7: Перебор вершин с отсеечениями по числу цветов

```

1  int vertex_backtrack(int v, vector<vector<int>>& edges,
2  vector<int> solution, int large_color) {
3      if (large_color >= best - 1)
4          return INF;
5      if (v == n) {
6          int cnt = 0;
7          vector<int> cnt_col(n);
8          for (int i = 0; i < n; i++) {
9              cnt_col[solution[i]]++;
10             if (cnt_col[solution[i]] == 1)
11                 cnt++;
12         }
13         best = min(best, cnt);
14         return cnt;
15     }
16
17     int min_ans = n;
18
19     if (v == 0) {
20         solution[v] = 0;
21         min_ans = vertex_backtrack(v + 1, edges,
22             solution, 0);
23     }
24     else {
25         for (int color = 0; color < best - 1; color++)
26             {
27                 solution[v] = color;

```

```

26         bool correct = 1;
27         for (int i = 0; i < edges[v].size(); i++) {
28             if (solution[edges[v][i]] == color)
29                 correct = 0;
30         }
31         if (correct) {
32             int cur_ans = vertex_backtrack(v + 1,
33                                             edges, solution,
34                                             max(color, large_color));
35             min_ans = min(min_ans, cur_ans);
36         }
37     }
38     return min_ans;
39 }

```

Алгоритм 4: Перебор с эвристикой DSatur

Листинг 4.8: Вспомогательная функция выбора следующей вершины

```

1  int next_vertex_dsatur(vector<vector<int>>& edges,
2      vector<set<int>>& sat, vector<int>& solution) {
3
4      int max_sat = -1, next_vertex = -1, max_deg = -1;
5
6      for (int i = 0; i < n; i++) {
7          if (solution[i] == -1) {
8              int curr_sat = sat[i].size();
9              int curr_deg = edges[i].size();
10
11              if (curr_sat > max_sat ||
12                  (curr_sat == max_sat && curr_deg > max_deg)) {
13                  max_sat = curr_sat;
14                  next_vertex = i;
15                  max_deg = curr_deg;
16              }
17          }
18      }
19      return next_vertex;
20 }

```

Листинг 4.9: Точный алгоритм с эвристикой DSatur

```

1      int dsatur_backtrack(vector<vector<int>>& edges, vector
2      <int> solution, vector<set<int>>& sat,
3      int large_color,
4      int colored_count) {
5          if (large_color >= best - 1)
6              return INF;
7
8          if (colored_count == n) {
9              int cnt = 0;
10             vector<int> cnt_col(n);
11             for (int i = 0; i < n; i++) {
12                 cnt_col[solution[i]]++;
13                 if (cnt_col[solution[i]] == 1)
14                     cnt++;
15             }
16             if (best > cnt) {
17                 best = min(best, cnt);
18                 //fix = solution;
19             }
20             return cnt;
21         }
22
23         int min_ans = n;
24
25         int v = next_vertex_dsatur(edges, sat, solution);
26
27         if (colored_count == 0) {
28             solution[v] = 0;
29             vector<set<int>> new_sat = sat;
30             for (int i : edges[v]) {
31                 new_sat[i].insert(0);
32             }
33             min_ans = dsatur_backtrack(edges, solution,
34                                     new_sat, 0, colored_count + 1);
35         }
36         else {
37             for (int color = 0; color < best - 1; color++)
38                 {
39                     bool correct = true;

```

```

38         for (int i = 0; i < edges[v].size(); i++) {
39             if (solution[edges[v][i]] == color) {
40                 correct = false;
41                 break;
42             }
43         }
44
45         if (correct) {
46             solution[v] = color;
47             vector<set<int>> new_sat = sat;
48             for (int i : edges[v]) {
49                 new_sat[i].insert(color);
50             }
51             int cur_ans = dsatur_backtrack(edges,
52                 solution, new_sat,
53                 max(color, large_color),
54                 colored_count + 1);
55             min_ans = min(min_ans, cur_ans);
56         }
57     }
58
59     return min_ans;
60 }

```

Алгоритм 5: Метод Зыкова

Листинг 4.10: Рекурсивная функция метода Зыкова

```

1  int zykov_method(vector<vector<bool>>& g) {
2      int n = g.size();
3
4      pair<int, int> res = find_non_adj(g);
5      int u = res.first;
6      int v = res.second;
7
8      if (u == -1 && v == -1) {
9          return n;
10     }
11 }

```

```

12         vector<vector<bool>> g1 = add_edge(g, u, v);
13         vector<vector<bool>> g2 = contract_vertices(g, u, v
14             );
15
16         int chi1 = zykov_method(g1);
17         int chi2 = zykov_method(g2);
18
19         return min(chi1, chi2);
20     }

```

Листинг 4.11: Вспомогательные функции для метода Зыкова

```

1     vector<vector<bool>> add_edge(vector<vector<bool>>& g,
2         int u, int v) {
3         vector<vector<bool>> new_g = g;
4         new_g[u][v] = 1;
5         new_g[v][u] = 1;
6
7         return new_g;
8     }
9
10    vector<vector<bool>> contract_vertices(vector<vector<
11        bool>>& g, int u, int v) {
12        int n = g.size();
13
14        vector<vector<bool>> new_g(n - 1, vector<bool>(n -
15            1, false));
16
17        int new_i = 0;
18        for (int i = 0; i < n; i++) {
19            if (i == v) continue;
20
21            int new_j = 0;
22            for (int j = 0; j < n; j++) {
23                if (j == v) continue;
24
25                if (i != j) {
26                    if (g[i][j]) {
27                        new_g[new_i][new_j] = true;
28                    }
29
30                    if ((i == u && g[v][j]) || (j == u && g

```

```

28         [i][v])) {
29             new_g[new_i][new_j] = true;
30         }
31     }
32     new_j++;
33 }
34 new_i++;
35 }
36
37 return new_g;
38 }
39
40 pair<int, int> find_non_adj(vector<vector<bool>>& g) {
41     int n = g.size();
42     for (int i = 0; i < n; i++) {
43         for (int j = i + 1; j < n; j++) {
44             if (!g[i][j]) {
45                 return { i, j };
46             }
47         }
48     }
49     return { -1, -1 };
50 }

```

Алгоритм 6: Recursive Largest First (RLF)

ЛИСТИНГ 4.12: Recursive Largest First (RLF)

```

1  int rlf_graph_coloring(int n, const vector<vector<bool>>&
2  adj, vector<int>& colors) {
3      int color = 1;
4      vector<bool> U(n, true);
5
6      for (int v = 0; v < n; v++) {
7          colors[v] = 0;
8      }
9
10     while (true) {
11         bool U_empty = true;

```

```

11     for (int i = 0; i < n; i++) {
12         if (U[i]) { U_empty = false; break; }
13     }
14     if (U_empty) break;
15
16     int v = -1;
17     int max_deg = -1;
18     for (int i = 0; i < n; i++) {
19         if (U[i]) {
20             int deg = 0;
21             for (int j = 0; j < n; j++) {
22                 if (U[j] && adj[i][j]) deg++;
23             }
24             if (deg > max_deg) {
25                 max_deg = deg;
26                 v = i;
27             }
28         }
29     }
30
31     vector<int> S;
32     S.push_back(v);
33
34     vector<int> A;
35     for (int i = 0; i < n; i++) {
36         if (U[i] && adj[v][i]) {
37             A.push_back(i);
38         }
39     }
40
41
42     vector<int> W;
43     for (int i = 0; i < n; i++) {
44         if (!U[i]) continue;
45         bool in_S = (i == v);
46         bool in_A = false;
47         for (int a : A) {
48             if (a == i) { in_A = true; break; }
49         }
50         if (!in_S && !in_A) {
51             W.push_back(i);

```

```

52     }
53 }
54
55
56 while (!W.empty()) {
57     vector<int> d_A(W.size(), 0);
58     int max_dA = -1;
59     for (int idx = 0; idx < (int)W.size(); idx++) {
60         int w = W[idx];
61         int count = 0;
62         for (int a : A) {
63             if (adj[w][a]) count++;
64         }
65         d_A[idx] = count;
66         if (count > max_dA) max_dA = count;
67     }
68
69
70     int w_star = -1;
71     int w_star_idx = -1;
72     vector<int> candidates;
73     for (int idx = 0; idx < (int)W.size(); idx++) {
74         if (d_A[idx] == max_dA) {
75             candidates.push_back(idx);
76         }
77     }
78
79     if (candidates.size() > 1) {
80         int best_idx = -1;
81         int best_deg = -1;
82         for (int idx : candidates) {
83             int w = W[idx];
84             int deg = 0;
85             for (int j = 0; j < n; j++) {
86                 if (U[j] && adj[w][j]) deg++;
87             }
88             if (deg > best_deg) {
89                 best_deg = deg;
90                 best_idx = idx;
91             }
92         }

```

```

93         w_star_idx = best_idx;
94         w_star = W[best_idx];
95     }
96     else {
97         w_star_idx = candidates[0];
98         w_star = W[candidates[0]];
99     }
100
101     S.push_back(w_star);
102
103     for (int x : W) {
104         if (adj[w_star][x]) {
105             bool already_in_A = false;
106             for (int a : A) {
107                 if (a == x) { already_in_A = true;
108                             break; }
109             }
110             if (!already_in_A) {
111                 A.push_back(x);
112             }
113         }
114
115         vector<int> new_W;
116         for (int x : W) {
117             if (x == w_star) continue;
118             if (adj[w_star][x]) continue;
119             new_W.push_back(x);
120         }
121         W = new_W;
122     }
123
124     for (int s : S) {
125         colors[s] = color;
126     }
127
128     for (int s : S) {
129         U[s] = false;
130     }
131
132     color++;

```

```
133         }
134
135         return color - 1;
136     }
```

Литература

- [1] Lewis R. M. R. A Guide to Graph Colouring: Algorithms and Applications. – Cham : Springer, 2016. – 253 p.
- [2] Алексеев В.Е., Таланов В.А. Графы. Модели вычислений. Структуры данных: Учебник. – Нижний Новгород: Изд-во ННГУ, 2005. 307 с.
- [3] Revista de Informática Teórica e Aplicada - RITA - ISSN 2175-2745 Vol. 25, Num. 04 (2018) 57-73.
- [4] Hasenplaugh W. et al. Ordering heuristics for parallel graph coloring //Proceedings of the 26th ACM symposium on Parallelism in algorithms and architectures. – ACM, 2014. – P. 166-177.
- [5] Brélaz, D. (1979). New methods to color the vertices of a graph. Communications of the ACM, 22(4), 251-256.
- [6] Fomin F. V., Kratsch D. Exact Exponential Algorithms. – Berlin : Springer, 2010. – 203 p. – ISBN 978-3-642-16532-0.
- [7] Корт Б., Файген Й. Комбинаторная оптимизация. Теория и алгоритмы : пер. с нем. – 3-е изд., испр. – М. : МЦНМО, 2019. – 720 с. – ISBN 978-5-4439-1375-5.
- [8] Vica's Blog. The Graph Coloring Problem (GCP) [Электронный ресурс]. – 2017. – <https://vicaws.github.io/blog/2017/graph-coloring>